

# ПЛАН ДЕМОНСТРАЦИИ ПРОГРАММНОГО ПРОДУКТА

Выпускная квалификационная работа

*Telegram-бот для автоматизированного учёта питания и расчёта КБЖУ*

## 1. Общие сведения о программном продукте

Программный продукт — Telegram-бот для ведения дневника питания с автоматическим расчётом калорийности, белков, жиров и углеводов (КБЖУ). Бот работает внутри мессенджера Telegram и не требует установки дополнительных приложений.

Стек: язык Go, база данных PostgreSQL, развёртывание через Docker Compose. Пользовательский интерфейс построен на inline-кнопках Telegram Bot API.

## 2. Цель демонстрации

Продемонстрировать комиссии работоспособность разработанного программного продукта в реальных условиях эксплуатации: показать полный цикл взаимодействия пользователя с ботом — от первого запуска до формирования недельного отчёта и редактирования записей.

## 3. Технические условия для проведения демонстрации

### 3.1. Требования к оборудованию и среде

- Устройство с установленным Telegram (смартфон или ПК с Telegram Desktop)
- Интернет-соединение для взаимодействия с Telegram Bot API
- Развёрнутый стек: бот и база данных запущены через `docker-compose up -d`
- Токен бота и параметры подключения к БД указаны в `config.yml` (файл не публикуется в репозитории)

### 3.2. Подготовка перед демонстрацией

1. Убедиться, что оба контейнера запущены: `docker ps` — должны отображаться `telegram-bot` и `telegram-bot-db`
2. Проверить логи на наличие строки «Bot started» — подключение к Telegram API подтверждено
3. Открыть бота в Telegram и убедиться, что он отвечает на `/start`
4. При необходимости очистить тестовые записи в базе данных, чтобы отчёт за сегодня начинался с нуля

## 4. Сценарии демонстрации

Демонстрация охватывает 10 последовательных этапов. Каждый этап соответствует одному пользовательскому сценарию, описанному в разделе 4.4 ВКР.

| № этапа | Действие / демонстрируемый сценарий        | Ожидаемый результат                                                                                        |
|---------|--------------------------------------------|------------------------------------------------------------------------------------------------------------|
| 1       | Запустить бота командой /start             | Приветственное сообщение, появляется кнопка «Меню»                                                         |
| 2       | Открыть меню командой /menu                | Отображаются inline-кнопки: «Отчёт за сегодня», «Отчёт за вчера», «Отчёт за неделю», «Редактировать»       |
| 3       | Ввести «Гречка 200» — точное совпадение    | Бот подтверждает: «Запись добавлена: Гречка (200 г)», показывает КБЖУ                                      |
| 4       | Ввести «Творог 150» — несколько совпадений | Бот выводит inline-кнопки: «Творог 2%», «Творог 5%», «Творог 9%». После выбора — подтверждение записи      |
| 5       | Ввести «Ласось 300» — опечатка             | Алгоритм Левенштейна находит «Люсось», запись сохраняется корректно                                        |
| 6       | Ввести название несуществующего продукта   | Сообщение: «Продукт не найден», запись не создаётся                                                        |
| 7       | Открыть «Отчёт за сегодня»                 | Список всех записей дня с суммарными КБЖУ                                                                  |
| 8       | Нажать «Редактировать» в отчёте            | Каждая запись превращается в кнопку удаления; удалить одну позицию — список обновляется в том же сообщении |
| 9       | Нажать «Закончить редактирование»          | Возврат к итоговому отчёту с кнопкой «Редактировать»                                                       |

## 5. Пояснения к ключевым сценариям

### 5.1. Нечёткий поиск (этапы 4–5)

При вводе «Творог 150» поиск проходит три шага: точное совпадение не найдено → поиск по подстроке возвращает три варианта → бот выводит inline-кнопки выбора. Это демонстрирует работу алгоритма поиска по подстроке.

При вводе «Ласось 300» первые два шага не дают результата → запускается расчёт расстояния Левенштейна (порог `maxDistance = 1`) → найден «Лосось» с дистанцией 1. Это демонстрирует устойчивость к опечаткам.

## 5.2. Редактирование без нового сообщения (этап 8)

Бот использует метод `EditMessageText` вместо отправки нового сообщения. При удалении записи сообщение обновляется на месте — чат не засоряется. Это архитектурное решение стоит отдельно прокомментировать комиссии.

## 5.3. Изоляция данных пользователей

Каждая запись в таблице `food_log` привязана к Telegram ID пользователя. Все SQL-запросы используют параметризованные выражения — данные одного пользователя недоступны другому, SQL-инъекции исключены.

## 6. Вероятные вопросы комиссии и краткие ответы

### Почему Go, а не Python?

Go компилируется в единый бинарный файл, горутины обрабатывают одновременные запросы без накладных расходов на потоки. Для Telegram-бота, где каждое сообщение — отдельный HTTP-запрос, это практично.

### Почему база данных в памяти (словарь продуктов), а не вторая таблица в PostgreSQL?

Список из ~75 позиций не меняется в ходе работы бота. Хранение в памяти исключает лишний SQL-запрос при каждом поиске. Архитектура допускает перенос в базу без изменений в других слоях — достаточно реализовать тот же интерфейс.

### Почему логи не сохраняются между перезапусками?

Текущая реализация выводит логи в `stdout` контейнера — это стандартная практика для Docker на этапе разработки. Для промышленной эксплуатации предусмотрено подключение системы централизованного хранения логов (например, Loki или ELK) как направление дальнейшего развития.

### Как защищены данные пользователей?

Изоляция по Telegram ID, параметризованные SQL-запросы, токен бота и пароль БД хранятся в `config.yml` вне репозитория. Внешний порт базы данных не пробрасывается — подключение к PostgreSQL возможно только из сети контейнеров.